SYSCRAFTERS 2026 – SYSTEM DESIGN HACKATHON

SALESTORM

High-Scale E-CommerceFlashSaleSystem

Team: StormForge

Team Members:

Dharsan S - 24AM020

Gokulavasan B - 24AM028

Manjunath S M - 24AM057

Ragul S - 24AM303

Problem & Requirements

The Challenge

- **10,000 customers** may click **Buy Now simultaneously**
- Only **100 units** are available
- System must prevent **overselling**
- Prevent **duplicate reservations and payments**
- Handle **payment/service failures**
- Maintain a correct **order lifecycle**

Key Goal

10,000 requests → Maximum 100 successful sales → 0 overselling

High-Level Architecture

SALESTORM Architecture

Flow:

Customers → CDN / WAF → Load Balancer → API Gateway
↓
Product, Cart, Sale → Inventory & Reservation → Checkout → Payment
↓
Order → Shipment → Notification

Supporting Components

- Redis Cache
- Message Queue
- D atab ase
- Monitoring & Logging

High Concurrency & Inventory Reservation

How We Prevent Overselling

Purchase Flow: Buy Now → Check Inventory → Reserve Stock → Payment → Confirm Reservation → Create Order

Reservation States

AVAILABLE → RESERVED → PAYMENT_PENDING →

CONFIRMED → SOLD

If payment fails or reservation expires: **RESERVED → RELEASED**

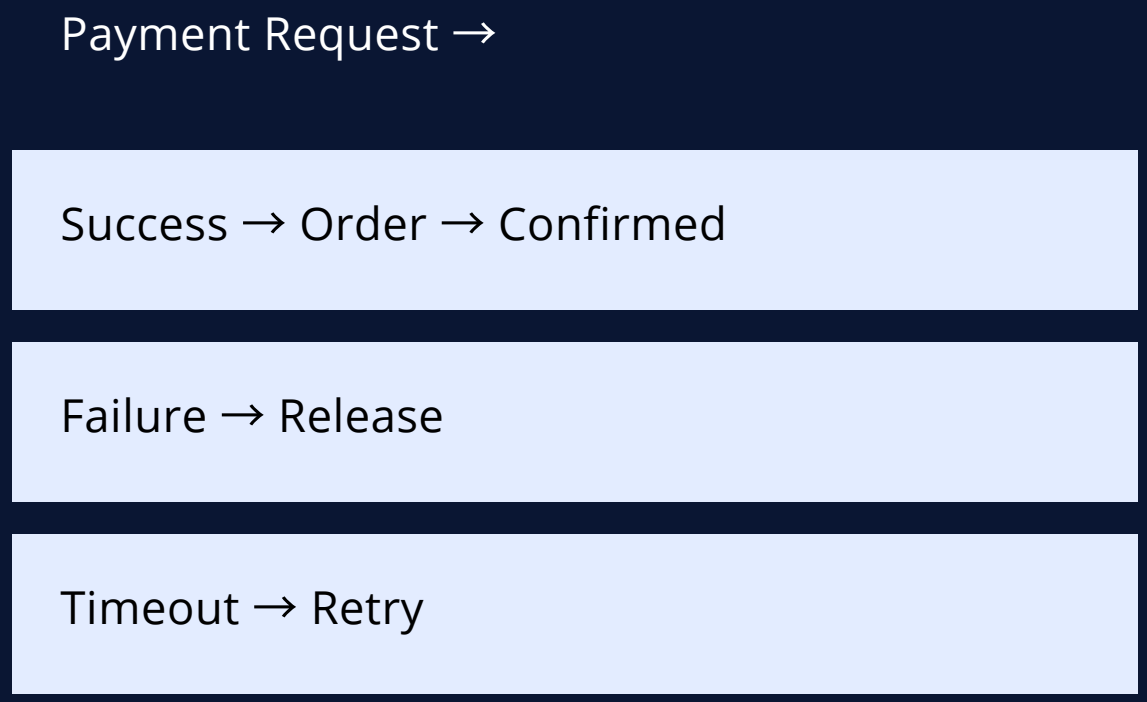
Key Protection

- Concurrency control
- Atomic inventory update
- Idempotency key
- Reservation expiry
- Maximum **100 successful sales**

Payment & Order Reliability

ReliablePaymentProcessing

Payment States: Payment Request branches to Success, Failure, Timeout. Success → Order → Confirmed. Failure → Release. Timeout → Retry.



Order Lifecycle

CREATED → PAYMENT_PENDING → CONFIRMED → PROCESSING → SHIPPED → OUT_FOR_DELIVERY → DELIVERED

Reliability

- Idempotent payment requests
- Timeout & retry
- Circuit breaker
- Failure recovery
- Compensation / reconciliation
- Duplicate request protection

Scalability, Security & SOLID

Scalability

- Horizontal scaling
- Load balancing
- Redis caching
- Message queues
- Database scaling
- Rate limiting

Security & Observability

- Authentication & authorization
- HTTPS
- Input validation
- Secure payment handling
- Audit logging
- Metrics, logs & distributed tracing

SOLID & Design Patterns

- **SRP** — Separate service responsibilities
- **OCP** — Easily add payment m eth o ds
- **DIP** — Depend on interfaces
- **Strategy** — Payment m eth o ds
- **State** — Order/Reservation lifecycle
- **Circuit Breaker** — Failure protection

Final Solution & Key Takeaway

Why SALESTORM Works

- Handles **10,000 concurrent purchase requests**
- Limits successful sales to **100 units**
- Prevents **overselling**
- Prevents **duplicate payments/orders**
- Supports **reservation expiry and recovery**
- Handles **payment and service failures**
- Supports **horizontal scaling**
- Provides **security and observability**